

Introdução à Computação redimensionando arrays

Alexandre Rademaker

arrays I

Um array é uma porção contínua na memória ocupada por segmentos de igual tamanho, logo mesmo tipo.

Quando criamos um array, definimos sua capacidade de armazenamento, isto é, seu tamanho.

Lembrem-se que em C tentar acessar uma posição além do limite do array não gera erro!

Mas e se no decorrer da computação precisamos de mais espaço?

arrays II

Temos duas possibilidades.

Alocar um espaço maior do que o necessário de início, eventualmente desperdiçando espaço. Mas precisamos definir o pior caso, o máximo que “podemos precisar”.

Quando o limite for atingido, criar um novo e maior array, copiar os valores para o novo array.

redimensionando arrays I

Usando alocação estática

list0.c

```
1 int list[3];
2 list[0] = 1; list[1] = 2; list[2] = 3;
3
4 // ops! I need 10 slots
5
6 int new[10];
7 for (int i = 0; i < 10; i++)
8     new[i] = i < 3 ? list[i] : 0;
```

redimensionando arrays II

Se quisermos alocar o array no **heap**

list1.c

```
1 int *list = malloc(3 * sizeof(int));
2 if (list == NULL) return 1;
3
4 list[0] = 1; list[1] = 2; list[2] = 3;
5
6 // ops! I need 10 slots
7
8 int *new = malloc(10 * sizeof(int));
9 if (new == NULL) { free(list); return 1; }
10
11 for (int i = 0; i < 10; i++)
12     new[i] = i < 3 ? list[i] : 0;
13
14 free(list); // important
15 list = new;
```

redimensionando arrays III

Usando alocação estática, não podemos renomear um array

```
list = new;
```

para isso precisamos de ponteiros. Em `list0.c` vejamos como a memória é ocupada.

Após transferir valores, com ponteiros, precisamos lembrar de liberar a memória antes de 'atualizar' o ponteiro.

redimensionando arrays I

Mas o processo pode ser simplificado com `realloc` da biblioteca `stdlib.h`, vide `man 3 realloc`.

```
void *realloc(void *ptr, size_t size);
```

O tipo `size_t` é definido como `unsigned integer`.

Mas atenção, as novas posições podem não ser 'zeradas'.

redimensionando arrays II

list2.c

```
1 int *list = malloc(3 * sizeof(int));
2 if (list == NULL)
3     return 1;
4
5 list[0] = 1; list[1] = 2; list[2] = 3;
6
7 list = realloc(list, 10 * sizeof(int));
8 if (list == NULL) {
9     free(list); return 1;
10 }
11
12 for (int i = 3; i < 10; i++)
13     list[i] = 0;
14
15 free(list); return 0;
```

operações com arrays I

Para recuperar o valor de índice n (acesso aleatório) no array, a complexidade é $O(1)$.

Se o array estiver ordenado, a complexidade de localizar se um valor existe no array é $O(\log n)$ (digamos que queremos adicionar um elemento que não exista).

E temos um tempo $O(n)$ para adicionar um novo valor (copiando um por um para uma nova região) se o array estiver 'lotado'.

O melhor caso, para inserção e pesquisa têm $\Omega(1)$, se encontrarmos o valor imediatamente na primeira posição, ou ter memória livre após nosso array para adicionar um novo valor.